

```
In [1]: import pandas as pd
import numpy as np

import seaborn as sns
import matplotlib as mpl
import matplotlib.pyplot as plt

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image

for i in [pd, np, mpl, sns]:
    print(i.__name__, i.__version__)
```

```
pandas 2.1.4
numpy 1.26.4
matplotlib 3.8.0
seaborn 0.12.2
```

1-3 데이터 변환

```
In [2]: Image('15.png')
```

Out[2]: **0. 데이터셋 소개**

Titanic

[Titanic](#) 탑승객의 생존 유무를 담은 데이터셋입니다.

Data	Description	Dictionary
PassengerId	Passenger Id, Index	
Survived	Survival	0 = No, 1 = Yes
Pclass	Ticket class	1 = 1st, 2 = 2nd, 3 = 3rd
Name	Name	
Sex	Sex	
Age	Age in years	
Sibsp	# of siblings / spouses aboard the Titanic	
Parch	# of parents / children aboard the Titanic	
Ticket	Ticket number	
Fare	Passenger fare	
Cabin	Cabin number	
Embarked	Port of Embarkation	C = Cherbourg, Q = Queenstown, S = Southampton

다양한 형식의 데이터를 가지고 있으며, 여러 가지 아이디어를 생각하고 시도할 만한 요소가 많게끔 기획된 데이터셋입니다.

```
In [2]: df_titanic = pd.read_csv('data/titanic.csv', index_col='PassengerId')
df_titanic
```

Out[2]:

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId											
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...
887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows × 11 columns

1. 수치형 데이터

In [3]: Image('16.png')

Out[3]: **정규화(Normalize)**

변수의 최소값을 0, 최대값이 1이 되도록 변환합니다.

$$X_{new} = \frac{X - \min(X)}{\max(X) - \min(X)}$$

sklearn.preprocessing.MinMaxScaler

주요 파라미터

파라미터명	기능
feature_range	(최소값, 최대값) 형태의 tuple

[Ex.1]

df_titanic에서 Age와 Fare의 최소값은 0, 최대값이 1이 되도록 변환합니다.

그리고 변환된 값을 변수명 뒤에 '_n'을 붙여 Age_n, Fare_n으로 저장합니다.

```
In [3]: from sklearn.preprocessing import MinMaxScaler

mm_scaler = MinMaxScaler()

# 정규화 대상 컬럼입니다.
mm_cols = ['Age', 'Fare']
# 정규화 변환기를 학습합니다.
mm_scaler.fit(df_titanic[mm_cols])
```

Out[3]: MinMaxScaler(copy=True, feature_range=(0, 1))

```
In [4]: '''
이 코드는 `sklearn.preprocessing` 모듈에서 제공하는
`MinMaxScaler`를 사용하여 주어진 데이터프레임 `df_titanic`의
특정 열들을 최소-최대 스케일링 (Min-Max Scaling)하여
정규화하는 작업을 수행합니다.

1. `mm_cols = ['Age', 'Fare']`:
   정규화할 대상 열(특성)의 리스트를 지정합니다.
   이 경우 'Age'와 'Fare' 열이 정규화 대상입니다.
2. `mm_scaler.fit(df_titanic[mm_cols])`:
   `fit` 메서드를 사용하여 데이터프레임 `df_titanic`의 'Age'와 'Fare' 열에 대한
   최소-최대 스케일링을 위한 변환기를 학습합니다.
   이 작업을 통해 스케일링에 사용될 최소값(min)과 최대값(max)을 결정합니다.
3. `mm_cols_n = [i + '_n' for i in mm_cols]`:
   정규화된 데이터를 저장할 새로운 열 이름을 만듭니다.
   기존 열 이름에 '_n'을 추가하여 새로운 열 이름을 지정합니다.
4. `mm_scaler.transform(df_titanic[mm_cols])`:
   `transform` 메서드를 사용하여 'Age'와 'Fare' 열을 최소-최대 스케일링합니다.
   이를 통해 해당 열의 데이터가 0에서 1 사이의 값으로 변환됩니다.
5. `pd.DataFrame(...):` `transform` 메서드로부터 반환된 정규화된 데이터를
   새로운 데이터프레임으로 변환합니다.
   이 데이터프레임은 인덱스를 기존 데이터프레임 `df_titanic`의 인덱스와 일치하며,
   새로운 열 이름인 `mm_cols_n`을 사용합니다.
6. `df_titanic[mm_cols_n] = ...`:
   정규화된 데이터프레임을 기존 데이터프레임 `df_titanic`에 추가합니다.
   여기서 `mm_cols_n`은 새로운 열 이름을 나타내며,
   해당 열에는 정규화된 데이터가 포함됩니다.

따라서 이 코드는 'Age'와 'Fare' 열을 최소-최대 스케일링하여 정규화한 후,
이를 기존 데이터프레임에 새로운 열로 추가하는 작업을 수행합니다.
'''

mm_cols_n = [i + '_n' for i in mm_cols]
# 복수의 numpy array를 DataFrame에 추가하려면, DataFrame 형태로 되어 있어야 합니다.
# DataFrame을 통해 추가시, 대상 Frame의 index로 인덱스를 구성합니다.
df_titanic[mm_cols_n] = \
    pd.DataFrame(mm_scaler.transform(df_titanic[mm_cols]),
                  index=df_titanic.index)
df_titanic[mm_cols_n].head()
```

Out[4]:

	Age_n	Fare_n
PassengerId		
1	0.271174	0.014151
2	0.472229	0.139136
3	0.321438	0.015469
4	0.434531	0.103644
5	0.434531	0.015713

In [5]: # 정규화 처리 후의 결과를 확인해봅니다.
df_titanic[mm_cols_n].agg(['min', 'max'])

Out[5]:

	Age_n	Fare_n
min	0.0	0.0
max	1.0	1.0

[Ex.2]

df_titanic에서 Age와 Fare의 최소값은 -1, 최대값이 1이 되도록 변환합니다.

그리고 변환된 값을 변수명 뒤에 '_n'을 붙여 Age_n, Fare_n으로 저장합니다.

In [6]: '''
이 코드는 `MinMaxScaler`를 사용하여 데이터프레임 `df\_titanic`의
특정 열을 최소-최대 스케일링하여 정규화합니다.
그리고 나서 최소화 및 최대화된 값의 범위를 확인하기 위해
각 열의 최솟값(min)과 최댓값(max)을 계산합니다.

1. `mm\_scaler = MinMaxScaler((-1, 1))`:
`MinMaxScaler`의 인스턴스를 생성합니다.

이때 ``feature_range=(-1, 1)`` 인자를 통해 최소-최대 스케일링된 값의 범위를 -1에서 1로 지정합니다. 기본적으로는 0에서 1로 정규화됩니다.

2. ``mm_scaler.fit_transform(df_titanic[mm_cols])``:
``fit_transform`` 메서드를 사용하여 'Age'와 'Fare' 열을 최소-최대 스케일링합니다. 이 작업을 통해 해당 열의 데이터가 -1에서 1 사이의 값으로 변환됩니다.
3. ``pd.DataFrame(...)``:
``fit_transform`` 메서드로부터 반환된 정규화된 데이터를 새로운 데이터프레임으로 변환합니다.
 이 데이터프레임은 인덱스를 기존 데이터프레임 ``df_titanic``의 인덱스와 일치하며, 새로운 열 이름인 ``mm_cols_n``을 사용합니다.
4. ``df_titanic[mm_cols_n] = ...``:
 정규화된 데이터프레임을 기존 데이터프레임 ``df_titanic``에 추가합니다.
 여기서 ``mm_cols_n``은 새로운 열 이름을 나타내며, 해당 열에는 정규화된 데이터가 포함됩니다.
5. ``df_titanic[mm_cols_n].agg(['min', 'max'])``:
 각 열의 최솟값(min)과 최댓값(max)을 확인하기 위해 ``agg`` 함수를 사용합니다.
``agg`` 함수는 지정된 집계 함수를 각 열에 적용하여 결과를 반환합니다.
 여기서는 최솟값과 최댓값을 확인하기 위해 ``min``과 ``max``를 집계 함수로 사용했습니다.

따라서 이 코드는 'Age'와 'Fare' 열을 최소-최대 스케일링하여 정규화한 후, 이를 기존 데이터프레임에 새로운 열로 추가하고, 최소화된 값과 최대화된 값을 확인하는 작업을 수행합니다.

MinMaxScaler의 매개변수로 최소값과 최대값을 지정할 수 있습니다.

```
mm_scaler = MinMaxScaler((-1, 1))
```

정규화 변환기를 학습후 바로 변환값을 뽑아냅니다.

그리고 컬럼을 추가합니다.

```
mm_scaler.fit_transform(df_titanic[mm_cols])
df_titanic[mm_cols_n] = \
    pd.DataFrame(mm_scaler.transform(df_titanic[mm_cols]),
                  index=df_titanic.index)
df_titanic[mm_cols_n].agg(['min', 'max'])
```

```
Out[6]:
```

	Age_n	Fare_n
min	-1.0	-1.0
max	1.0	1.0

```
In [4]: Image('17.png')
```

```
Out[4]: 표준화(Standardize)
```

- 변수의 평균이 0, 표준편차가 1이 되도록 변환합니다.

$$X_{std} = \frac{X - \mu}{\sigma}$$

sklearn.preprocessing.StandardScaler

scipy.stats.zscore

※ Remark: `sklearn.preprocessing.StandardScaler` 는 편향(biased) 표준편차를 사용합니다.

`sklearn.preprocessing.StandardScaler` 가이드에서 발췌한 내용

We use a biased estimator for the standard deviation, equivalent to ``numpy.std(x, ddof=0)``. Note that the choice of ``ddof`` is unlikely to affect model performance.

[Ex.3]

sklearn.preprocessing.StandardScaler를 통해 df_titanic에서 Age와 Fare를 표준화합니다.
그리고 변환된 값을 변수명 뒤에 '_s'을 붙여 Age_s, Fare_s으로 저장합니다.

In [7]:

```
'''
이 코드는 `StandardScaler`를 사용하여 데이터프레임 `df_titanic`의
특정 열을 표준화(평균 0, 표준편차 1)하는 작업을 수행하고,
표준화된 값들의 평균과 표준편차를 확인합니다.

1. `std_scaler = StandardScaler()` :
   `StandardScaler`의 인스턴스를 생성합니다.
   이것은 평균을 0으로 하고 표준편차를 1로 스케일링하는데 사용됩니다.
2. `std_cols = ['Age', 'Fare']` :
   표준화할 대상 열(특성)의 리스트를 지정합니다.
   이 경우 'Age'와 'Fare' 열이 표준화 대상입니다.
3. `std_cols_s = [i + '_s' for i in std_cols]` :
   표준화된 데이터를 저장할 새로운 열 이름을 만듭니다.
   각 원래 열 이름에 '_s'를 추가하여 새로운 열 이름을 만듭니다.
4. `std_scaler.fit_transform(df_titanic[std_cols])` :
   `fit_transform` 메서드를 사용하여 'Age'와 'Fare' 열을 표준화합니다.
   이 작업을 통해 해당 열의 데이터가 평균이 0이고
   표준편차가 1인 값으로 변환됩니다.
5. `pd.DataFrame(...)` :
   `fit_transform` 메서드로부터 반환된 표준화된 데이터를
   새로운 데이터프레임으로 변환합니다.
   이 데이터프레임은 인덱스를 기존 데이터프레임
   `df_titanic`의 인덱스와 일치하며,
   새로운 열 이름인 `std_cols_s`을 사용합니다.
6. `pd.concat(..., axis=1)` :
   `pd.concat` 함수를 사용하여 데이터프레임을 연결합니다.
   이때 연결하는 대상은 평균과 표준편차를 나타내는
   데이터프레임입니다. `axis=1`은 열 방향으로 연결함을 의미합니다.
7. `df_titanic[std_cols_s].mean().rename('mean')` :
   표준화된 데이터프레임의 각 열에 대해 평균을 계산하고,
   `rename` 메서드를 사용하여 열 이름을 'mean'으로 변경합니다.
8. `df_titanic[std_cols_s].std(ddof=0).rename('std')` :
```

표준화된 데이터프레임의 각 열에 대해 표준편차를 계산하고,
 `rename` 메서드를 사용하여 열 이름을 'std'로 변경합니다.
 `ddof=0`은 표본 표준편차를 계산할 때 편향 보정을 하지 않음을 나타냅니다.

따라서 이 코드는 'Age'와 'Fare' 열을 표준화하여 새로운 열로 추가하고,
 표준화된 데이터의 평균과 표준편차를 확인하는 작업을 수행합니다.
 ...

```
from sklearn.preprocessing import StandardScaler
std_scaler = StandardScaler()
std_cols = ['Age', 'Fare']
std_cols_s = [i + '_s' for i in std_cols]
df_titanic[std_cols_s] = \
    pd.DataFrame(std_scaler.fit_transform(df_titanic[std_cols]),
                  index=df_titanic.index)
pd.concat([
    df_titanic[std_cols_s].mean().rename('mean'),
    df_titanic[std_cols_s].std(ddof=0).rename('std')
], axis=1)
```

Out[7]:

	mean	std
Age_s	2.174187e-16	1.0
Fare_s	-4.373606e-17	1.0

- `scipy.stats.zscore`는 `ddof`를 통해 편향(biased, 모)

불편향(unbiased, 표본) 표준편차를 적용할지를 정할 수 있습니다.

[Ex.4]

Fare를 표준화합니다. 표준화시 불편향(unbiased) 표준편차를 사용합니다.
 변환된 값은 `Fare_s2`에 저장합니다.

In [8]:

```
'''
코드는 파이썬의 SciPy 라이브러리에서 `zscore` 함수를 사용하여
```

Titanic 데이터셋의 'Fare' 열에 대한 Z 점수를 계산하는 것입니다.
Z 점수는 통계적으로 데이터 포인트가 평균에서 얼마나 떨어져 있는지를 나타내는 표준화된 값입니다.

여기서 `zscore` 함수는 각 데이터 포인트의 Z 점수를 계산하는데,
Z 점수는 해당 데이터 포인트가 그 열의 평균에서 몇 표준편차만큼 떨어져 있는지를 나타냅니다.
이를 통해 데이터를 평균이 0이고 표준편차가 1인 표준 정규 분포로 표준화할 수 있습니다.

코드에서는 'Fare' 열의 Z 점수를 계산하고, 그 결과를 'Fare_s2' 열에 저장합니다.
또한, `agg` 함수를 사용하여 'Fare_s2' 열의 평균과 표준편차를 계산합니다.
'mean'은 평균을, 'std'는 표준편차를 나타내는 값을 반환합니다.
이렇게 하면 데이터의 평균과 표준편차를 쉽게 파악할 수 있습니다.
'''

```
from scipy.stats import zscore
```

```
df_titanic['Fare_s2'] = zscore(df_titanic['Fare'], ddof=1)
df_titanic['Fare_s2'].agg(['mean', 'std'])
```

```
Out[8]: mean    -1.196200e-17
      std     1.000000e+00
      Name: Fare_s2, dtype: float64
```

정규화와 표준화의 효과

- 회귀 분석시 정규화와 표준화를 통해 변수의 스케일을 균일화하여,
독립변수가 종속변수에 미치는 정도를 파악하기 쉽게 합니다.
- Ridge와 Lasso와 같은 모수의 크기에 대해 제약을 가하는 모델에서
규제가 균일하게 적용되도록 합니다.
- Gradient Descent와 같은 공간 탐색 기반의 알고리즘에서
학습률과 변수들간의 영향이 균일하도록 합니다.

In [3]: `Image('18.png')`

Out[3]: **거듭제곱 변환(Power Transformation)**

- 변수의 분포를 우측 또는 좌측으로 치우친 정도를 조정합니다.
- 거듭수를 나타내는 파라미터 λ 에 따라 변환함수가 달라집니다.

λ	거듭 제곱 함수
$\lambda \neq 0$	$X_{new} = X^\lambda$
$\lambda = 0$	$X_{new} = \log(X)$

- 변환 효과

λ	효과
$\lambda = 0$ (log 변환)	우측 꼬리의 길이를 줄여, 우측으로 치우치게 분포하도록 변환합니다.
$\lambda > 1$	우측 꼬리의 길이를 늘립니다. λ 가 커질수록 우측 꼬리의 길이가 커집니다.
$0 < \lambda < 1$	우측 꼬리의 길이를 줄여서 우측으로 치우친 형태의 분포로 변환 됩니다.
$\lambda < 0$	역수로 변환 됩니다.

[Ex.5]

df_titanic $\lambda=[-2, -1, -0.5, 0, 0.5, 1, 1.5, 2]$ 에 따라

Power Transform된 Fare의 분포를 히스토그램으로 출력해봅니다.

In [9]: `# Fare의 기술 통계를 봅니다.
df_titanic['Fare'].describe().to_frame().T`

```
Out[9]:
```

	count	mean	std	min	25%	50%	75%	max
Fare	891.0	32.204208	49.693429	0.0	7.9104	14.4542	31.0	512.3292

※ Remark

λ 가 0인 경우 log변환을 하게되는데 Fare의 최소값이 0이므로,
Fare가 0인 경우가 존재합니다. 이 때 log 변환은 성립하지 않게 됩니다.
이 때, log의 값이 미지수가 되어 버립니다.
이럴 경우 임의의 수를 더하여 0을 log로 취하는 것을 방지합니다.
변환 후 최소값이 0이 되도록 $\log(X + 1)$ 로 변환합니다.

```
In [10]: '''
이 코드는 Matplotlib와 Seaborn 라이브러리를 사용하여
Titanic 데이터셋의 'Fare' 열에 대해 Box-Cox 변환을 수행한 후
변환된 데이터의 히스토그램을 시각화합니다.

여기서 Box-Cox 변환은 데이터의 정규성을 증가시키기 위해 사용되며,
변환의 정도는  $\lambda$ (람다) 값에 따라 달라집니다.
이 코드에서는 여러 가지  $\lambda$  값에 대해 변환을 수행하고,
각  $\lambda$  값에 대한 히스토그램을 시각화합니다.

- `fig, axes = plt.subplots(2, 4, figsize=(12, 6))`:
  2x4 격자 모양의 subplot을 생성하고,
  해당 subplot들을 나타내는 figure 객체와 axes 배열을 반환합니다.
  그리고 이 subplot들의 전체적인 크기를 설정합니다.
- `lams = [-2, -1, -0.5, 0, 0.5, 1, 1.5, 2]`:
  변환할 때 사용할  $\lambda$  값의 리스트를 정의합니다.
- `for lam, ax in zip(lams, axes.ravel())`:
   $\lambda$  값과 subplot을 순회합니다.
- `if lam != 0`:  $\lambda$  값이 0이 아니면,
  - `sns.histplot(df_titanic['Fare'] ** lam, ax=ax)`:
```

```

        해당  $\lambda$  값으로 'Fare' 열을 Box-Cox 변환하여 히스토그램을 그립니다.
    - `else:`:  $\lambda$  값이 0이면,
      - `sns.histplot(np.log(df_titanic['Fare'] + 1), ax=ax)`:
        로그 변환 후 히스토그램을 그립니다.
    - `ax.set_title('lambda={}'.format(lam))`:
      subplot의 제목을 설정합니다.
- `plt.tight_layout()`:
  subplot들 간의 간격을 조정하여 레이아웃을 최적화합니다.
- `plt.show()`:
  시각화된 subplot들을 화면에 표시합니다.

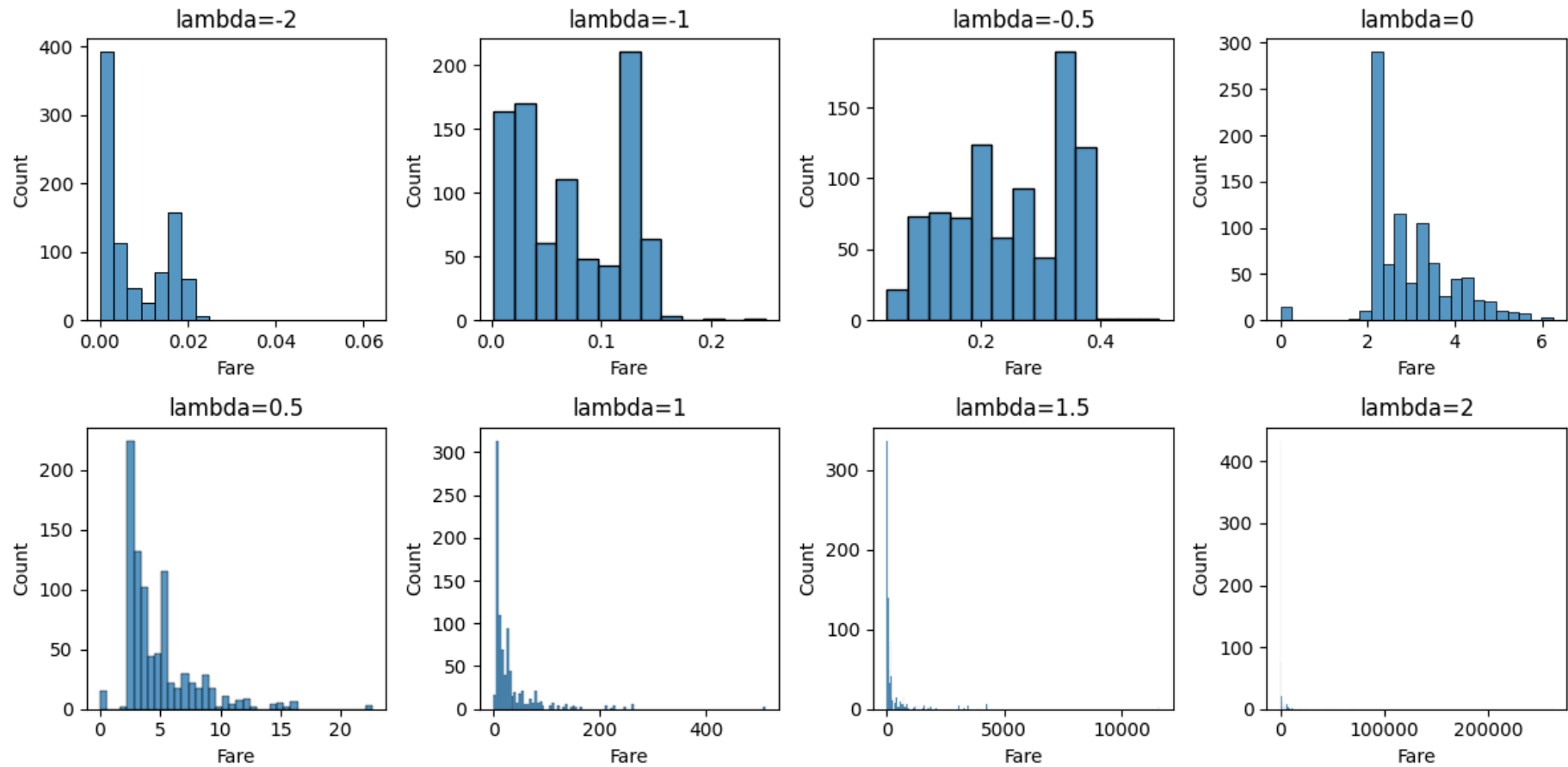
```

이렇게 하면 λ 값에 따라 다른 변환을 수행한 데이터의 분포를 한눈에 확인할 수 있습니다.

```

fig, axes = plt.subplots(2, 4, figsize=(12, 6))
lams = [-2, -1, -0.5, 0, 0.5, 1, 1.5, 2]
for lam, ax in zip(lams, axes.ravel()):
    if lam != 0:
        sns.histplot(df_titanic['Fare'] ** lam, ax=ax)
    else:
        sns.histplot(np.log(df_titanic['Fare'] + 1), ax=ax)
    ax.set_title('lambda={}'.format(lam))
plt.tight_layout()
plt.show()

```



[Ex.6]

df_titanic $\lambda = [-2, -1, -0.5, 0, 0.5, 1, 1.5, 2]$ 에 따라

Power Transform된 Age의 분포를 히스토그램으로 출력해봅니다.

이 때 Age의 결측치는 제외합니다.

```
In [11]: # Fare의 기술 통계를 봅니다.
```

```
df_titanic['Age'].describe().to_frame().T
```

```
Out[11]:
```

	count	mean	std	min	25%	50%	75%	max
Age	714.0	29.699118	14.526497	0.42	20.125	28.0	38.0	80.0

```
In [12]: '''
이 코드는 Python에서 Matplotlib와 Seaborn을 사용하여
타이타닉 데이터셋을 시각화하는 것입니다.
주어진 데이터셋은 타이타닉호 탑승자들의 정보를 포함하고 있습니다.

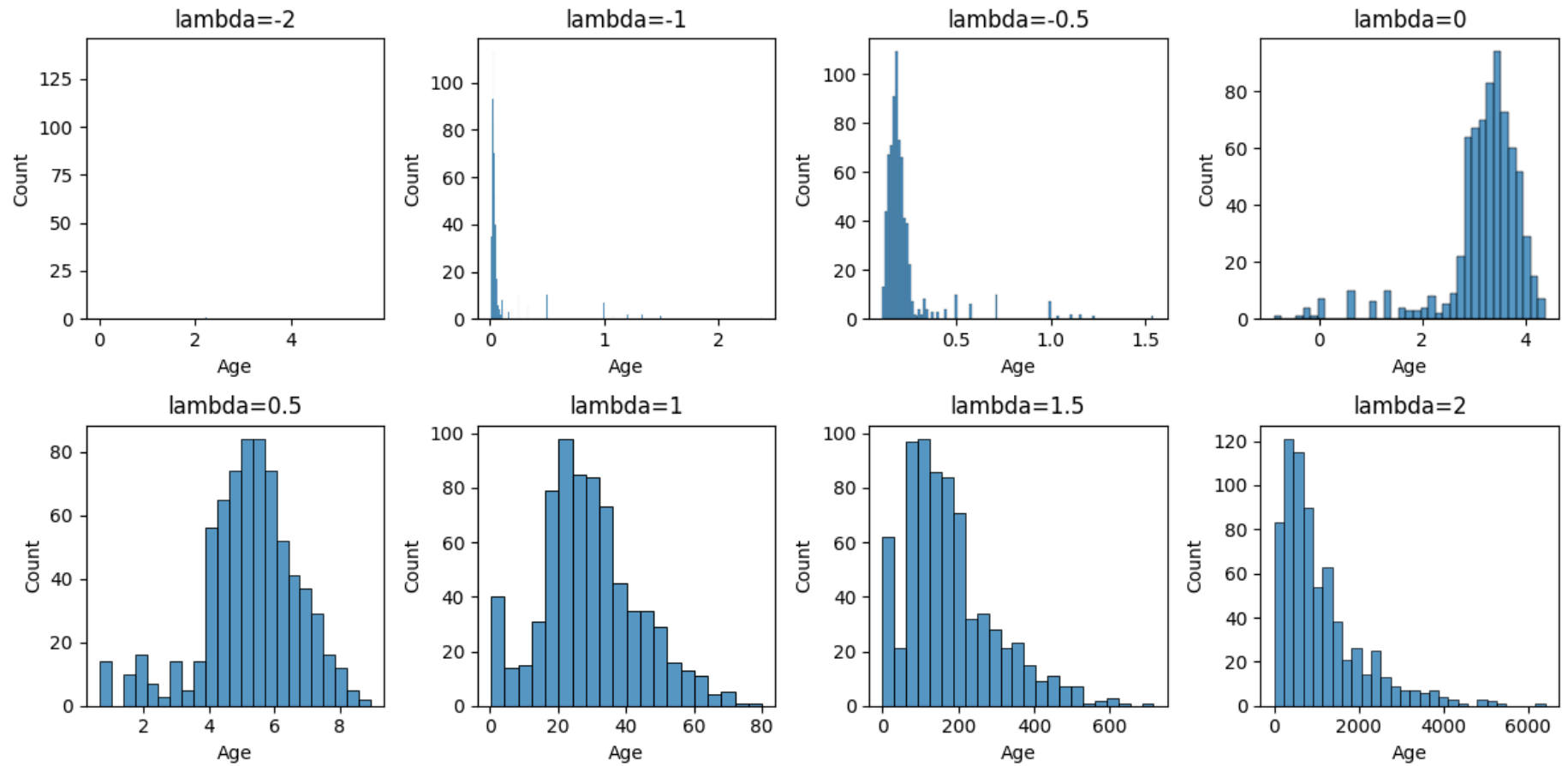
1. `fig, axes = plt.subplots(2, 4, figsize=(12, 6))`:
   이 부분은 2x4 격자 모양의 서브플롯을 생성합니다.
   각각의 subplot은 `axes` 변수에 저장됩니다.
   전체 그림의 크기는 12x6 인치입니다.
2. `lams = [-2, -1, -0.5, 0, 0.5, 1, 1.5, 2]`:
   람다 값의 리스트를 정의합니다. 여기서 람다 값은 변환에 사용됩니다.
3. `s_age = df_titanic.loc[df_titanic['Age'].notna(), 'Age']`:
   타이타닉 데이터셋에서 나이 정보가 있는 행들의
   'Age' 열을 선택하여 `s_age`에 저장합니다.
4. `for lam, ax in zip(lams, axes.ravel())`:
   `lams` 리스트의 각 람다 값에 대해 반복하면서, `axes`의 각 subplot에 접근합니다.
5. `if lam != 0`: 현재 람다 값이 0이 아니라면,
   - `sns.histplot(s_age ** lam, ax=ax)`:
     나이 데이터를 현재의 람다 값으로 변환한 후,
     Seaborn의 `histplot` 함수를 사용하여 히스토그램을 그립니다.
6. `else`: 람다 값이 0인 경우,
   - `sns.histplot(np.log(s_age), ax=ax)`:
     나이 데이터를 자연로그로 변환한 후,
     Seaborn의 `histplot` 함수를 사용하여 히스토그램을 그립니다.
7. `ax.set_title('lambda={}'.format(lam))`:
   각 subplot에 람다 값에 대한 제목을 설정합니다.
8. `plt.tight_layout()`:
   서브플롯 간의 간격을 조정하여 레이아웃을 더 깔끔하게 만듭니다.
9. `plt.show()`:
   모든 서브플롯을 표시합니다.

이 코드는 다양한 람다 값에 따라 나이 데이터의 변환된 분포를 시각화합니다.
람다 값이 양수인 경우 데이터가 오른쪽으로 치우쳐진 형태로 변환되며,
음수인 경우 데이터가 왼쪽으로 치우쳐진 형태로 변환됩니다.
'''
```


람다 값이 0이면 자연로그 변환이 적용됩니다.

'''

```
fig, axes = plt.subplots(2, 4, figsize=(12, 6))
lams = [-2, -1, -0.5, 0, 0.5, 1, 1.5, 2]
s_age = df_titanic.loc[df_titanic['Age'].notna(), 'Age']
for lam, ax in zip(lams, axes.ravel()):
    if lam != 0:
        sns.histplot(s_age ** lam, ax=ax)
    else:
        sns.histplot(np.log(s_age), ax=ax)
    ax.set_title('lambda={}'.format(lam))
plt.tight_layout()
plt.show()
```



In [4]: `Image('19.png')`

Out[4]: **Box-Cox 변환(Box-Cox Transformation)**

- Power Transformation을 기반으로한 변환으로 정규 분포에 가까운 형태(정규 분포를 따르게 되는 것은 보장하지 않습니다.)로 변환합니다.
- λ 에 따라 변환함수가 달라집니다. 데이터에서 정규분포와 가깝게 해주는 λ 를 찾아줍니다

λ	거듭 제곱 함수
$\lambda \neq 0$	$X^{(\lambda)} = \frac{X^\lambda - 1}{\lambda}$
$\lambda = 0$	$X_{new} = \log(X)$

- λ 는 아래의 손실 함수를 사용하여 구해집니다.

$$l(\lambda) = -\frac{n}{2} \ln \left(\frac{1}{n} \sum_{j=1}^n (x_j^{(\lambda)} - \bar{x}_j^{(\lambda)})^2 \right) + (\lambda - 1) \sum_{j=1}^n x_j$$

손실 함수를 최소화하는 λ 를 구하여 변환 합니다.

```
sklearn.preprocessing.PowerTransformer, method='box-cox'
```

[Ex.7]

df_titanic $\lambda=[-2, -1, -0.5, 0, 0.5, 1, 2]$ 에 따라 Power Transform된

Age의 분포를 히스토그램으로 출력해봅니다. 이 때 Age의 결측치는 제외합니다.

여기에 box-cox Transformation의 결과까지 추가합니다.

```
In [13]: '''
이 코드는 sklearn의 PowerTransformer를 사용하여 데이터를 변환하고,
변환된 데이터의 히스토그램을 Seaborn을 통해 시각화합니다.
여전히 타이타닉 데이터셋을 다루고 있습니다.
```

```

1. `from sklearn.preprocessing import PowerTransformer`:
   sklearn 라이브러리에서 PowerTransformer를 가져옵니다.
   이는 데이터 변환을 위한 클래스입니다.
2. `fig, axes = plt.subplots(2, 4, figsize=(12, 6))`:
   이 부분은 2x4 격자 모양의 서브플롯을 생성합니다.
   각각의 subplot은 `axes` 변수에 저장됩니다.
   전체 그림의 크기는 12x6 인치입니다.
3. `lams = ['box-cox', -2, -1, -0.5, 0, 0.5, 1, 2]`:
   변환에 사용할 람다 값들을 포함한 리스트를 정의합니다.
   여기서 'box-cox'는 Box-Cox 변환을 의미합니다.
4. `pt = PowerTransformer(method='box-cox')`:
   PowerTransformer 객체를 생성합니다.
   이 때, 변환 방법(method)으로 'box-cox'를 선택합니다.
5. `for lam, ax in zip(lams, axes.ravel())`:
   `lams` 리스트의 각 람다 값에 대해 반복하면서,
   `axes`의 각 subplot에 접근합니다.
6. `if lam == 'box-cox':`: 람다 값이 'box-cox'인 경우,
   - `sns.histplot(pt.fit_transform(s_age.to_frame())[:, 0], ax=ax)`:
     Box-Cox 변환을 적용한 후, 변환된 데이터의 히스토그램을 그립니다.
   - `ax.set_title('box-cox, lambda={:.5}'.format(pt.lambdas_[0]))`:
     subplot의 제목을 Box-Cox 변환에 대한 것으로 설정하고,
     해당 변환에 사용된 람다 값을 포함합니다.
7. `elif lam != 0:`: 람다 값이 0이 아닌 경우,
   - `sns.histplot(s_age ** lam, ax=ax)`:
     나이 데이터를 현재의 람다 값으로 변환한 후,
     Seaborn의 `histplot` 함수를 사용하여 히스토그램을 그립니다.
   - `ax.set_title('lambda={}'.format(lam))`:
     subplot의 제목을 현재 람다 값에 대한 것으로 설정합니다.
8. `else:`: 람다 값이 0인 경우,
   - `sns.histplot(np.log(s_age), ax=ax)`:
     나이 데이터를 자연로그로 변환한 후,
     Seaborn의 `histplot` 함수를 사용하여 히스토그램을 그립니다.
   - `ax.set_title('lambda={}'.format(lam))`:
     subplot의 제목을 0에 대한 것으로 설정합니다.
9. `plt.tight_layout()`:
   서브플롯 간의 간격을 조정하여 레이아웃을 더 깔끔하게 만듭니다.
10. `plt.show()`:
    모든 서브플롯을 표시합니다.

```

이 코드는 Box-Cox 변환과 다른 람다 값을 사용한 데이터 변환의 결과를 시각화합니다.
Box-Cox 변환은 최적의 람다 값을 자동으로 선택하여 변환을 수행하며,

다른 람다 값은 해당 값에 따라 데이터가 어떻게 변환되는지를 보여줍니다.

'''

```
from sklearn.preprocessing import PowerTransformer
```

```
fig, axes = plt.subplots(2, 4, figsize=(12, 6))
```

```
lams = ['box-cox', -2, -1, -0.5, 0, 0.5, 1, 2]
```

```
pt = PowerTransformer(method='box-cox')
```

```
for lam, ax in zip(lams, axes.ravel()):
```

```
    if lam == 'box-cox':
```

```
        sns.histplot(pt.fit_transform(s_age.to_frame())[0], ax=ax)
```

```
        ax.set_title('box-cox, lambda={:.5}'.format(pt.lambdas_[0]))
```

```
    elif lam != 0:
```

```
        sns.histplot(s_age ** lam, ax=ax)
```

```
        ax.set_title('lambda={}'.format(lam))
```

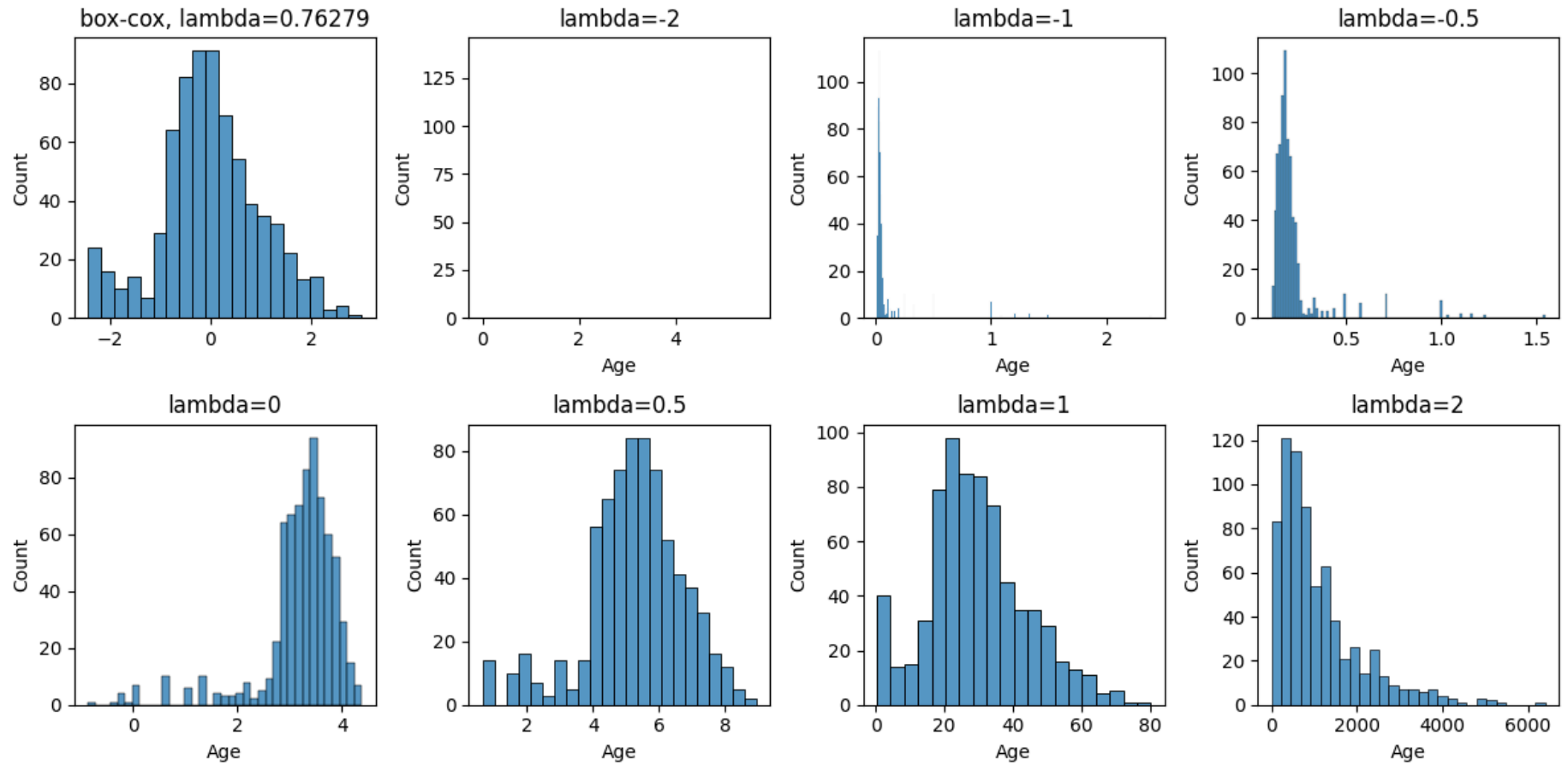
```
    else:
```

```
        sns.histplot(np.log(s_age), ax=ax)
```

```
        ax.set_title('lambda={}'.format(lam))
```

```
plt.tight_layout()
```

```
plt.show()
```



2. 범주형 데이터

In [5]: `Image('20.png')`

Out[5]: **가변수화(One-Hot Encoding, 지시변수화)**

범주형 변수를 모델에 포함시키기 위해, 각 수준(범주)마다 범주에 해당 여부를 나타내는 하나의 이진 변수를 할당시킵니다.

예를들어, apple, banana, peach 범주들로 구성된 변수 **fruit**을 가변수해봅니다.

fruit	fruit_apple	fruit_banana	fruit_peach
apple	1	0	0
banana	0	1	0
peach	0	0	1
...			
apple	1	0	0

sklearn.preprocessing.OneHotEncoder, pd.get_dummies

[Ex.8]

df_titanic에서 Pclass와 Embarked를 가변수화 합니다.

Embarked의 결측은 최빈값으로 대체합니다.

가변수화한 후 새로운 데이터프레임 을 만듭니다.

In [14]: '''
이 코드는 "Embarked" 열의 결측값을 해당 열의 가장 빈번한 값으로 채우는 것입니다.
타이타닉 데이터셋에서 "Embarked" 열은 승객이 탑승한 항구를 나타냅니다.

1. `df_titanic['Embarked'].mode()`:`
"Embarked" 열에서 가장 빈번하게 나타나는 값을 찾기 위해
`mode()` 함수를 사용합니다. 이는 결측값을 채울 때 사용될 값입니다.
2. `df_titanic['Embarked'].iloc[0]`:`

`mode()` 함수의 결과는 `Series` 형식이므로,
가장 빈번한 값을 가져오기 위해 `iloc`를 사용하여 첫 번째 값을 선택합니다.

3. ``df_titanic['Embarked'].fillna()`:`
"Embarked" 열의 결측값을 채우기 위해 `fillna()` 함수를 사용합니다.
이 함수는 인자로 전달된 값을 사용하여 결측값을 대체합니다.

4. ``df_titanic['Embarked'].fillna(df_titanic['Embarked'].mode().iloc[0])`:`
결측값을 "Embarked" 열에서 가장 빈번한 값으로 채웁니다.

따라서, 이 코드는 "Embarked" 열의 결측값을 해당 열에서
가장 빈번한 값으로 대체하여 데이터의 완결성을 보완합니다.
'''

```
df_titanic['Embarked']
= df_titanic['Embarked'].fillna(df_titanic['Embarked'].mode().iloc[0])
```

`pd.get_dummies`로 해봅시다. 새로운 데이터프레임을 `df_titanic_gd`로 합니다.

```
In [15]: # pd.get_dummies columns 파라미터에 가변수화를 할 컬럼들의 리스트를 전달합니다.
df_titanic_gd = pd.get_dummies(df_titanic, columns=['Pclass', 'Embarked'])
df_titanic_gd.head()
```


Out[15]:

	Survived	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Age_n	Fare_n	Age_s	Fare_s	Fa
PassengerId														
1	0	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	-0.457653	-0.971698	-0.530377	-0.502445	-0.5
2	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	-0.055542	-0.721729	0.571831	0.786845	0.7
3	1	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	-0.357125	-0.969063	-0.254825	-0.488854	-0.4
4	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	-0.130937	-0.792711	0.365167	0.420730	0.4
5	0	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	-0.130937	-0.968575	0.365167	-0.486337	-0.4

sklearn.preprocessing.OneHotEncoder로 해봅니다.

새로운 데이터프레임을 df_titanic_ohe로 합니다.

```
In [16]: from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(
    # pd.DataFrame에 넣기 위해 Dense Matrix 형태로 변환시킵니다.
```

```
sparse=False
)
ohe_cols = ['Pclass', 'Embarked']
ohe.fit(df_titanic[ohe_cols])
```

```
Out[16]: OneHotEncoder(categorical_features=None, categories=None, drop=None,
                        dtype=<class 'numpy.float64'>, handle_unknown='error',
                        n_values=None, sparse=False)
```

```
In [17]: # 가변수 변환을 해봅니다.
ohe.transform(df_titanic[ohe_cols])
```

```
Out[17]: array([[0., 0., 1., 0., 0., 1.],
                [1., 0., 0., 1., 0., 0.],
                [0., 0., 1., 0., 0., 1.],
                ...,
                [0., 0., 1., 0., 0., 1.],
                [1., 0., 0., 1., 0., 0.],
                [0., 0., 1., 0., 1., 0.]])
```

```
In [18]: '''
이 코드는 sklearn의 OneHotEncoder를 사용하여
범주형 변수를 원-핫 인코딩하는 것입니다.
타이타닉 데이터셋에서 "Pclass"와 "Embarked" 열은 범주형 변수입니다.

1. `from sklearn.preprocessing import OneHotEncoder`:
   sklearn 라이브러리에서 OneHotEncoder를 가져옵니다.
   이는 범주형 변수를 원-핫 인코딩하기 위한 클래스입니다.
2. `ohe = OneHotEncoder(sparse=False)`:
   OneHotEncoder 객체를 생성합니다. 여기서 `sparse=False`로 설정되어 있으므로,
   밀집 형태의 행렬을 반환합니다.
   기본적으로 OneHotEncoder는 희소 행렬을 반환합니다.
3. `ohe_cols = ['Pclass', 'Embarked']`:
   원-핫 인코딩할 열의 이름을 리스트로 지정합니다.
4. `ohe.fit(df_titanic[ohe_cols])`:
   OneHotEncoder를 데이터에 맞추습니다.
   이는 인코딩을 위해 각 열의 범주들을 학습하는 과정입니다.
5. `ohe.transform(df_titanic[ohe_cols])`:
   지정된 열을 원-핫 인코딩하여 변환합니다.
   이는 입력 데이터프레임의 해당 열을 원-핫 인코딩된 형태로 변환합니다.
6. `ohe.get_feature_names(ohe_cols)`:
```

인코딩된 각 특성의 이름을 반환합니다.
 여기서는 "Pclass"와 "Embarked" 열의 각 범주에 대한
 원-핫 인코딩된 특성들의 이름을 반환합니다.

이 코드는 "Pclass"와 "Embarked" 열을 원-핫 인코딩하여
 새로운 특성을 생성합니다. 이는 범주형 변수를 숫자형 변수로 변환하여
 머신러닝 알고리즘이 이해할 수 있도록 합니다.
 ...

가변수들의 명칭을 얻어옵니다.
 ohe.get_feature_names(ohe_cols)

```
Out[18]: array(['Pclass_1', 'Pclass_2', 'Pclass_3', 'Embarked_C', 'Embarked_Q',
               'Embarked_S'], dtype=object)
```

In [19]: ...
 위 코드는 타이타닉 데이터셋을 가공하는 과정을 나타냅니다.
 이 코드는 원-핫 인코딩(One-Hot Encoding)을 사용하여
 범주형 변수를 가변수(dummy variable)로 변환하고,
 그 결과를 기존의 데이터프레임에 추가하는 작업을 수행합니다.

1. ``pd.concat(..., axis=1)``:
 - ``pd.concat`` 함수는 데이터프레임을 이어 붙이는데 사용됩니다.
 - 여기서는 두 개의 데이터프레임을 이어 붙이는데 활용됩니다.
2. ``df_titanic.drop(columns=['Pclass', 'Embarked'])``:
 - ``df_titanic`` 데이터프레임에서 'Pclass'와 'Embarked' 열을 제외합니다.
 - ``drop`` 메서드를 사용하여 특정 열을 삭제합니다.
3. ``pd.DataFrame(ohe.transform(df_titanic[ohe_cols]), index=df_titanic.index, columns=ohe.get_feature_names(ohe_cols))``:
 - ``ohe.transform(df_titanic[ohe_cols])``:
 ``ohe_cols``에 해당하는 열을 원-핫 인코딩합니다.
 이를 통해 범주형 변수를 이진수 형태로 표현합니다.
 - ``pd.DataFrame(...)``:
 위에서 얻은 인코딩된 결과를 데이터프레임으로 변환합니다.
 - ``index=df_titanic.index``:
 새로운 데이터프레임의 인덱스를 기존의
 데이터프레임과 동일하게 설정합니다.
 - ``columns=ohe.get_feature_names(ohe_cols)``:
 새로운 데이터프레임의 열 이름을 원-핫 인코딩된

특성의 이름으로 설정합니다.

4. `pd.concat(..., axis=1)` (이어지는 부분):

- 이전에 제거한 열을 제외한 기존 데이터프레임과,
새롭게 만든 원-핫 인코딩된 데이터프레임을 이어 붙입니다.

이렇게 하면 기존의 데이터프레임에 있는 범주형 변수들이
원-핫 인코딩된 형태로 확장된 데이터프레임을 얻을 수 있습니다.
...

```
df_titanic_ohe = pd.concat([
    # Pclass와 Embarked를 제외합니다 - 제외 안해도 상관 없습니다.
    # get_dummies와 동일한 결과를 유도하기 위함입니다.
    df_titanic.drop(columns=['Pclass', 'Embarked']),
    pd.DataFrame(ohe.transform(df_titanic[ohe_cols]),
                  index=df_titanic.index,
                  columns=ohe.get_feature_names(ohe_cols))
], axis=1)
df_titanic_ohe.head()
```

Out[19]:

	Survived	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Age_n	Fare_n	Age_s	Fare_s	Fa
PassengerId														
1	0	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	-0.457653	-0.971698	-0.530377	-0.502445	-0.5
2	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	-0.055542	-0.721729	0.571831	0.786845	0.7
3	1	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	-0.357125	-0.969063	-0.254825	-0.488854	-0.4
4	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	-0.130937	-0.792711	0.365167	0.420730	0.4
5	0	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	-0.130937	-0.968575	0.365167	-0.486337	-0.4

In [20]:

```
'''
위 코드는 두 개의 데이터프레임인 `df_titanic_gd`와 `df_titanic_ohe`를
비교하고, 각 행에서 값이 서로 다른 셀의 개수를 계산합니다.

1. `df_titanic_gd.dropna()` :
- `dropna()` 메서드를 사용하여 `df_titanic_gd` 데이터프레임에서
  결측치가 있는 행을 제거합니다.
- 결측치가 있는 행을 제거하므로 데이터프레임의 행 수가 줄어듭니다.
'''
```

```

2. `df_titanic_ohe.dropna()` :
- `dropna()` 메서드를 사용하여 `df_titanic_ohe` 데이터프레임에서
  결측치가 있는 행을 제거합니다.
- 결측치가 있는 행을 제거하므로 데이터프레임의 행 수가 줄어듭니다.

3. `df_titanic_gd.dropna() != df_titanic_ohe.dropna()` :
- `!=` 연산자를 사용하여 두 데이터프레임을 요소별로 비교합니다.
- 이 연산은 두 데이터프레임의 각 셀을 비교하고,
  값이 다른 경우에는 True(1)로, 같은 경우에는 False(0)으로 반환합니다.

4. `.sum()` :
- 각 열의 값을 합하여 총 불일치 개수를 계산합니다.
- True 값(1)은 불일치를 나타내므로, 불일치하는 값의 총 개수가 반환됩니다.

따라서 위 코드는 두 데이터프레임을 비교하여
값이 다른 셀의 개수를 구하는 작업을 수행합니다.
결과적으로는 두 데이터프레임 간의 불일치하는 값의 총 개수가 반환됩니다.
'''

# 동일 여부를 확인해 봅니다.
# 결측치인 경우는 동일 유무가 비교가 되지 않으므로 제외합니다.
(df_titanic_gd.dropna() != df_titanic_ohe.dropna()).sum()

```

```
Out[20]: Survived      0
          Name        0
          Sex         0
          Age         0
          SibSp       0
          Parch       0
          Ticket      0
          Fare        0
          Cabin       0
          Age_n       0
          Fare_n      0
          Age_s       0
          Fare_s      0
          Fare_s2     0
          Pclass_1    0
          Pclass_2    0
          Pclass_3    0
          Embarked_C  0
          Embarked_Q  0
          Embarked_S  0
          dtype: int64
```

레이블 인코딩(Label Encoding)

범주형 변수들의 범주마다 고유의 숫자로 치환하는 처리법입니다.

sklearn.preprocessing.LabelEncoder

```
In [6]: Image('21.png')
```

Out[6]: 예를들어, apple, banana, peach 범주들로 구성된 범주형 변수 fruit를, 레이블 인코딩해봅니다.

fruit		fruit_l
apple		0
banana	▷	1
peach	apple→0, banana→1, peach→2	2
...		
apple		0

sklearn.preprocessing.LabelEncoder

[Ex.9]

df_titanic의 Embarked를 LabelEncoding을 해봅니다.

Embarked의 결측은 최빈값으로 대체합니다.

```
In [21]: '''
위 코드는 'Embarked' 열의 결측치를 해당 열의
최빈값(mode)으로 채우는 작업을 수행합니다.

1. `df_titanic['Embarked'].mode()`:
   - 'Embarked' 열의 최빈값을 찾습니다. 최빈값은 가장 자주 등장하는 값으로,
     여기서는 결측치를 채우기 위한 대체값으로 활용됩니다.
   - `.mode()` 함수를 사용하여 최빈값을 찾습니다.

2. `.fillna(df_titanic['Embarked'].mode().iloc[0])`:
   - `.fillna()` 메서드를 사용하여 결측치를 채웁니다.
   - 여기서는 최빈값으로 결측치를 채우기 위해
     `.mode().iloc[0]`를 사용합니다.
```


- ``iloc[0]``는 최빈값의 첫 번째 값을 가져옵니다.
최빈값이 여러 개일 경우에도 첫 번째 값을 선택합니다.
- 결측치를 최빈값으로 채워넣어 `'Embarked'` 열의 데이터를 수정합니다.

이렇게 하면 `'Embarked'` 열의 결측치가 해당 열의 최빈값으로 대체되어 데이터의 완전성을 유지할 수 있습니다.
'''

```
df_titanic['Embarked']
= df_titanic['Embarked'].fillna(df_titanic['Embarked'].mode().iloc[0])
```

In [22]:

```
'''
위 코드는 `sklearn.preprocessing` 모듈에서 제공하는
`LabelEncoder`를 사용하여 범주형 변수를 정수로 인코딩하는 과정을 나타냅니다.

1. `from sklearn.preprocessing import LabelEncoder`:
   - `sklearn.preprocessing` 모듈에서 `LabelEncoder` 클래스를 가져옵니다.
   - `LabelEncoder`는 범주형 변수를 정수로 변환하는데 사용됩니다.
2. `le = LabelEncoder()` :
   - `LabelEncoder` 클래스의 인스턴스를 생성합니다.
   - 이 인스턴스는 범주형 변수를 인코딩하는데 사용됩니다.
3. `le.fit(df_titanic['Embarked'])` :
   - `fit()` 메서드를 사용하여 `LabelEncoder`를 데이터에 맞춥니다.
   - 이 경우, 'Embarked' 열의 고유한 범주들을 찾아내어
     각 범주에 대해 정수를 할당할 준비를 합니다.
4. `le.transform(df_titanic['Embarked'])` :
   - `transform()` 메서드를 사용하여 실제로 데이터를 변환합니다.
   - 이 메서드는 'Embarked' 열의 각 범주를 해당하는 정수로 변환합니다.
   - 즉, 'Embarked' 열의 범주형 값을 정수로 인코딩하여 반환합니다.

이렇게 하면 `LabelEncoder`를 사용하여 'Embarked' 열의
범주형 데이터를 정수로 변환할 수 있습니다.
이는 주로 머신러닝 모델에 입력하기 전에 범주형 데이터를 처리할 때 사용됩니다.
'''
```

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
# 보통 transformer는 복수의 변수를 입력을 받습니다.
# 하지만, LabelEncoder는 단일 변수를 받습니다.
```

```
# 이 점에서 LabelEncoder의 목적이 주로  
# 수치형 대상값을 요구하는 모델을 위해 설계됐음을 알 수 있습니다.  
le.fit(df_titanic['Embarked'])  
le.transform(df_titanic['Embarked'])
```

```

Out[22]: array([2, 0, 2, 2, 2, 1, 2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 1, 2, 2, 0, 2, 2,
1, 2, 2, 2, 0, 2, 1, 2, 0, 0, 1, 2, 0, 2, 0, 2, 2, 0, 2, 2, 0, 0,
1, 2, 1, 1, 0, 2, 2, 2, 0, 2, 0, 2, 2, 0, 2, 2, 0, 2, 2, 2, 0, 0,
2, 2, 2, 2, 2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 0, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1,
2, 0, 2, 2, 0, 2, 1, 2, 0, 2, 2, 2, 0, 2, 2, 0, 1, 2, 0, 2, 0, 2,
2, 2, 2, 0, 2, 2, 2, 0, 0, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 0, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 0, 2,
2, 0, 2, 2, 0, 2, 2, 2, 2, 1, 2, 1, 2, 2, 2, 2, 0, 0, 1, 2,
1, 2, 2, 2, 0, 2, 2, 2, 0, 1, 0, 2, 2, 2, 2, 1, 0, 2, 2, 0, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0, 1,
2, 2, 0, 1, 2, 2, 2, 2, 2, 2, 2, 2, 0, 0, 2, 0, 2, 1, 2, 2, 2,
1, 2, 2, 2, 2, 2, 2, 0, 1, 2, 2, 2, 1, 2, 1, 2, 2, 2, 0,
2, 2, 2, 1, 2, 0, 0, 2, 2, 0, 0, 2, 2, 0, 1, 1, 2, 1, 2, 0, 0,
0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 0, 2, 2, 1, 2, 2, 0, 2, 2, 2, 0,
1, 2, 2, 2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
0, 2, 0, 2, 2, 2, 1, 1, 2, 0, 0, 2, 1, 2, 0, 0, 1, 0, 0, 2, 2, 0,
2, 0, 2, 0, 0, 2, 0, 0, 2, 2, 2, 2, 2, 1, 0, 2, 2, 2, 0, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 1, 2, 2, 2, 2,
2, 2, 0, 1, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 0, 2, 2, 2, 0, 0, 2, 0, 2, 2, 2, 1, 2, 2,
2, 2, 2, 2, 2, 1, 0, 2, 2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
0, 2, 2, 0, 2, 2, 2, 2, 0, 2, 0, 0, 2, 2, 2, 2, 1, 1, 2, 2, 0,
2, 2, 2, 2, 1, 2, 2, 0, 2, 2, 2, 1, 2, 2, 2, 2, 0, 0, 0, 1, 2, 2,
2, 2, 2, 0, 0, 0, 2, 2, 2, 0, 2, 0, 2, 2, 2, 0, 2, 2, 0, 2, 2,
0, 2, 1, 0, 2, 2, 0, 0, 2, 2, 1, 2, 2, 2, 2, 2, 2, 0, 2, 2, 2,
2, 1, 2, 2, 2, 0, 2, 2, 0, 2, 0, 0, 2, 2, 0, 2, 2, 2, 0, 2, 1,
2, 2, 2, 0, 0, 2, 2, 2, 0, 2, 2, 2, 0, 2, 2, 2, 1, 1, 2, 2,
2, 2, 2, 0, 2, 0, 2, 2, 2, 1, 2, 2, 1, 2, 2, 0, 2, 2, 2, 2,
2, 2, 2, 0, 2, 2, 0, 0, 2, 0, 2, 2, 2, 2, 2, 1, 1, 2, 2, 1, 2, 0,
2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0, 1, 0,
2, 2, 2, 0, 2, 2, 2, 2, 0, 2, 0, 2, 2, 2, 1, 0, 2, 0, 2, 0, 1,
2, 2, 2, 2, 2, 0, 0, 2, 2, 2, 2, 2, 0, 2, 1, 2, 2, 2, 2, 2, 2,
2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 1, 2, 0, 1, 2, 0, 2, 0, 2, 2, 0,
2, 2, 2, 0, 2, 2, 0, 0, 2, 2, 2, 0, 2, 0, 2, 2, 0, 2, 2, 2, 2,

```

```
0, 0, 2, 2, 2, 2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 0, 0, 2, 2, 2, 0,
2, 2, 2, 2, 2, 1, 2, 2, 2, 0, 1])
```

```
In [23]: '''
`le.classes_`는 `LabelEncoder` 객체가 변환한 범주형 변수의
고유한 클래스(범주)를 나타냅니다.
이 속성은 `fit()` 메서드를 호출한 후에만 사용할 수 있습니다.

여기서 각 요소를 살펴보겠습니다:

- `le`: LabelEncoder 객체입니다.
- `classes_`: LabelEncoder가 변환한 범주형 변수의
  고유한 클래스(범주)를 나타내는 속성입니다.

예를 들어, 'Embarked' 열의 범주형 변수를 정수로 인코딩했다면,
`le.classes_`는 'Embarked' 열의 고유한 범주(예: 'C', 'Q', 'S')를
포함하는 배열 또는 리스트를 반환합니다.
이 정보는 해당 변수의 어떤 정수가 어떤 범주를
나타내는지 알 수 있도록 도와줍니다.
'''

le.classes_
```

```
Out[23]: array(['C', 'Q', 'S'], dtype=object)
```

```
In [ ]:
```